Day 8 – State Management with Signals

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **IDE Ready**: Open VS Code. Prepare to write a state service using Angular Signals.

2. **Prerequisites**:
   - You must have completed Day 7 successfully (configuring authentication headers, interceptors, and guards).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain the concept of reactive state management and why Angular Signals improve rendering performance.
- Create and initialize writeable signals using the `signal()` constructor.
- Define read-only derived signals using `computed()`.
- Use the `effect()` API to schedule side effects on signal updates.
- Build a centralized shopping cart state service using Signals and share it across multiple components.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define reactive state tracking. Compare zone-based change detection with fine-grained Signals.
2. Key Concepts & Core Ideas	7 min	Explain Signal values, read-only computed derivations, and side-effect handlers.
3. Live Coding Walkthrough	23 min	Write the CartService using Signals, add item push/pop methods, update catalog cards, and display cart headers.
4. Check for Understanding	5 min	Q&A on signal mutability, computed evaluation caching, and zone-free rendering futures.

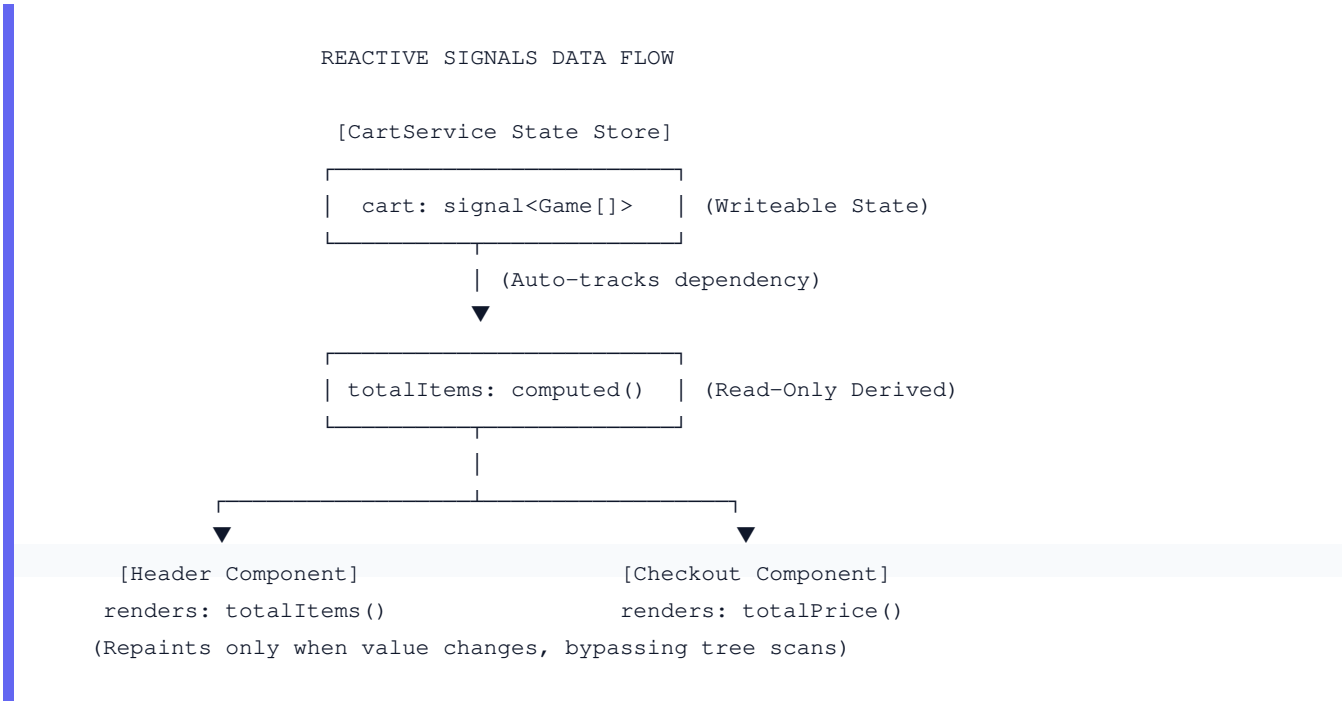
5. Class Wrap-up & Checkpoint	3 min	Verify cart quantities and prices update dynamically across components.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

In classic Angular, state changes rely on a library called `Zone.js`. When an event occurs (like clicking a button), Zone.js scans the entire component tree from top to bottom, checking if any variables have changed to repaint the HTML. This is called dirty checking. For large applications, this top-down scan is slow. Angular 22 LTS uses **Signals**—a reactive system that tells Angular exactly which specific DOM node needs to be repainted when a value changes.

- **Global State Sync**: We will build an active shopping cart. Clicking "Add to Cart" in the Catalog list component must update the cart item count in our header navigation bar, and update the total price in our cart review component, all updated in real-time.



2. Key Concepts & Core Ideas (7 min)

Introduce these reactive state concepts:

- **Signal**: A wrapper around a value that notifies interested consumers (components or other signals) whenever that value changes.
- **Writeable Signal**: A signal whose value can be set or updated directly. Methods include:
  - `set(newVal)`: Replaces the current value with a new one.

    - `update(fn)`: Computes a new value based on the previous value (e.g. `this.count.update(c => c + 1)`).

- **Computed Signal (`computed()`)**: A read-only signal that derives its value from other signals. It is lazily evaluated and cached; it only re-evaluates if the dependencies it reads actually change.
- **Effect (`effect()`)**: An operation that runs in response to signal updates. Useful for side effects like logging, storing state in localStorage, or drawing canvas charts.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Creating the Cart State Service

- **Concept**: Create a service `CartService` under `src/app/services/cart.service.ts`. We will define a writeable signal containing cart items, and computed signals for item counts and total costs.
- **Code**: Generate service:

```
ng generate service services/cart
```

Modify `src/app/services/cart.service.ts`:

```
import { Injectable, signal, computed, effect } from '@angular/core';
import { Game } from './game.service';

@Injectable({
  providedIn: 'root'
})
export class CartService {
  // 1. Initialize a writeable signal containing an empty array of Game objects
  private cartItems = signal<Game[]>([]);

  // 2. Expose the signal as a read-only computed value
  cart = computed(() => this.cartItems());

  // 3. Define a computed signal that computes total cart items
  totalItems = computed(() => this.cartItems().length);

  // 4. Define a computed signal that computes total cart cost
  totalPrice = computed(() => {
    return this.cartItems().reduce((sum, item) => sum + Number(item.price), 0);
  });

  constructor() {
    // 5. Setup an effect to save cart contents to localStorage automatically
    effect(() => {
      const items = this.cartItems();
      localStorage.setItem('cart_items', JSON.stringify(items));
      console.log(`[Cart State] Updated: ${items.length} items logged.`);
    });
```

```
    // Load initial state from localStorage if available
    const saved = localStorage.getItem('cart_items');
    if (saved) {
      this.cartItems.set(JSON.parse(saved));
    }
  }

  # Add game to cart
  addToCart(game: Game): void {
    // Use update() to append a game to the existing array safely
    this.cartItems.update(items => [...items, game]);
  }

  # Remove game from cart
  removeFromCart(gameId: number): void {
    this.cartItems.update(items => items.filter(item => item.id !== gameId));
  }

  # Clear all cart items
  clearCart(): void {
    this.cartItems.set([]); // Reset the signal value to an empty array
  }
}
```

Step 3.2: Updating Catalog View to Add Items

- **Concept**: Modify `GameListComponent` to inject `CartService` and add buttons allowing users to push items onto the cart state.
- **Code**: Modify `src/app/components/game-list/game-list.component.ts` (inject `CartService` and add catalog trigger):

```
import { CartService } from '../../services/cart.service'; // Import cart service
...
export class GameListComponent implements OnInit {
  ...
  constructor(
    private gameService: GameService,
    private cartService: CartService # Inject cart service
  ) {}
  ...
  addToCart(game: Game): void {
    this.cartService.addToCart(game);
  }
}
```

Modify `src/app/components/game-list/game-list.component.html` (inside the `@for` loop of games, add "Add to Cart"):

```
...
```

```
<li class="game-item">
  <span class="title">{{ game.title }}</span>
  <span class="price">${{ game.price | number:'1.2-2' }}</span>

  <a [routerLink]="['/games', game.id]" class="btn-link">View</a>
  <!-- Trigger cart state addition on click -->
  <button (click)="addToCart(game)" class="btn-cart">Add to Cart</button>
  ...
```

Step 3.3: Displaying Cart Summary in the Header

- **Concept**: Retrieve computed values in `AppComponent` to display the active shopping cart count in real-time.
- **Code**: Modify `src/app/app.component.ts`:

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
import { CartService } from './services/cart.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  // Inject the CartService. We will access its signals directly in the HTML template
  constructor(public cartService: CartService) {}
}
```

Modify `src/app/app.component.html` to output computed signals:

```
<header>
  <h1>Welcome to GameKey Store</h1>
  <nav>
    <a routerLink="/games" routerLinkActive="active-link">Catalog Inventory</a>
    <a routerLink="/games/add" routerLinkActive="active-link">Add Game</a>

    <!-- Call computed signals in template using function evaluation syntax () -->
    <div class="cart-summary">
        Cart: <strong>{{ cartService.totalItems() }}</strong> items
      (Total: <strong>${{ cartService.totalPrice() | number:'1.2-2' }}</strong>)
    </div>
  </nav>
</header>
<router-outlet></router-outlet>
```

- **Verify**: Open `http://localhost:4200/`. Click "Add to Cart" on different games. Notice that the cart total

and item count in the header update immediately. Refresh the page; check that your cart contents persist due to the `effect()` storing items in localStorage.

---

4. Check for Understanding (5 min)

> ***Question**: "Why do we write `cartService.totalItems()` (with parentheses) in the HTML template instead of `cartService.totalItems`?"*
> ***Answer**: In Angular, a Signal is a function wrapper. To read its value, you must invoke the function by adding parentheses `()`. This acts as a getter, allowing Angular to record that the active component template depends on this signal.*

> ***Question**: "What is the difference between a computed signal and a standard method getter like `get totalPrice()`?"*
> ***Answer**: Standard getters execute every single time change detection runs (which happens on mouse movement, clicks, or timeouts). A computed signal is lazily evaluated and cached. It computes the value once, and returns that cached value on subsequent scans. It only executes its logic again if its dependencies (the signals it reads) change.*

> ***Question**: "Why is it unsafe to update other signals inside the body of a `computed()` signal?"*
> ***Answer**: Computed signals are meant to be pure, read-only derivations. If a computed signal changes other signal states, it triggers cascading re-evaluations, which can lead to infinite loops and render crashes.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ The `CartService` is created and models state using writeable/computed signals.
  - ○ Adding items to the cart updates the catalog layout and header totals in real-time.
  - ○ The `effect()` API successfully persists state data to localStorage on changes.